

COS 214 Practical 4

TaskForge: Emergency Response Coordination System

Nicole Bare
Jezelle Govender
Rudolph

Repository: <https://github.com/<org>/Prac4-COS214>
Due 8 September 2026

1. System description

TaskForge is an Emergency Response Coordination System. It coordinates emergency units (fire engines, ambulances, police units and rescue units) during incidents such as building collapses and veld fires. Units are organised into a nested command hierarchy (Incident Command, sections, strike teams, individual units), move through a mission lifecycle (Standby, En Route, On Scene, Stabilizing, Transporting, Done), can be given extra capabilities at runtime (K9, aerial, technical rescue), and the hierarchy can be traversed in several ways to answer questions such as which units are available or where every fire engine is.

The four required Gang of Four patterns map directly onto these needs: Composite for the command hierarchy, State for the unit lifecycle, Decorator for runtime capabilities and Iterator for traversal. The full domain description and problem mapping from Task 1 follows.

Task 1: Domain description and GoF participant mapping

Task 1- A Quick Guide:

1. Domain Description

Emergency Response Coordination System (ERCS) is a system that coordinates emergency units (fire engines, ambulances, police, rescue teams) during incidents like wildfires, floods and building collapses.

The Problem It Solves

Problem	How ERCS Solves It
Units are organized hierarchically	Composite - Incident Command → Sections → Teams → Individual Units
Units change behavior during missions	State - Units transition: STANDBY → EN ROUTE → ON SCENE → STABILIZING → DONE
Add extra capabilities to units at runtime	Decorator - Wrap units with K9, Aerial, or Technical Rescue capabilities
Need to traverse the hierarchy in different ways	Iterator - Find all units, available units, or filter by type

2. The GoF Participant Mapping:

Composite Pattern

GoF Role Our Class

Component EmergencyComponent (abstract base)

Leaf EmergencyUnit, FireEngine, Ambulance, PoliceUnit, RescueUnit

Composite EmergencyGroup

State Pattern

GoF Role Our Class

Context EmergencyUnit (holds current state)

State UnitState (abstract)

ConcreteState StandbyState, EnRouteState, OnSceneState, StabilizingState, TransportingState, DoneSta

Decorator Pattern

GoF Role Our Class

Component EmergencyComponent

ConcreteComponent Any EmergencyUnit or EmergencyGroup instance

Decorator UnitDecorator (abstract)

ConcreteDecorator K9Decorator, AerialDecorator, TechnicalRescueDecorator

Iterator Pattern

GoF Role Our Class

Iterator EmergencyIterator (abstract)

ConcreteIterator AllUnitsIterator, AvailableUnitsIterator, TypeFilterIterator

Aggregate EmergencyComponent (declares factory methods)

ConcreteAggregate EmergencyGroup, EmergencyUnit

3. Important Design & Ownership Decisions:

Decision 1: Single EmergencyGroup (Not 3 Subclasses)

One EmergencyGroup class that can contain any EmergencyComponent (another group OR a unit).

Why: True recursion. A group can contain another group, which can contain another group... This is more faithful to the Composite pattern than hard-coding 3 levels.

Example:

```
EmergencyGroup("Incident Command") ← Level 1
├─ EmergencyGroup("Operations") ← Level 2
│   └─ EmergencyGroup("StrikeTeam") ← Level 3
│       └─ FireEngine("Engine 1") ← Level 4
```

Decision 2: States as Shared Singletons

Each state is a singleton (getInstance()). Units just point to the shared instance.

Why: States have no instance data, so they can be shared. This prevents memory leaks and double-deletion risks.

// How units use states:

```
unit->setState(StandbyState::getInstance());
```

```
unit->setState(EnRouteState::getInstance());
```

Decision 3: Executor/Advance/Cancel as Shared Operations

Three core operations that work across ALL patterns:

Operation	On Composite	On Leaf	On Decorator
execute()	Loops over children	Delegates to State	Delegates + adds own behavior
advance()	Loops over children	Tells State to transition	Delegates
cancel()	Loops over children	Tells State to cancel	Delegates

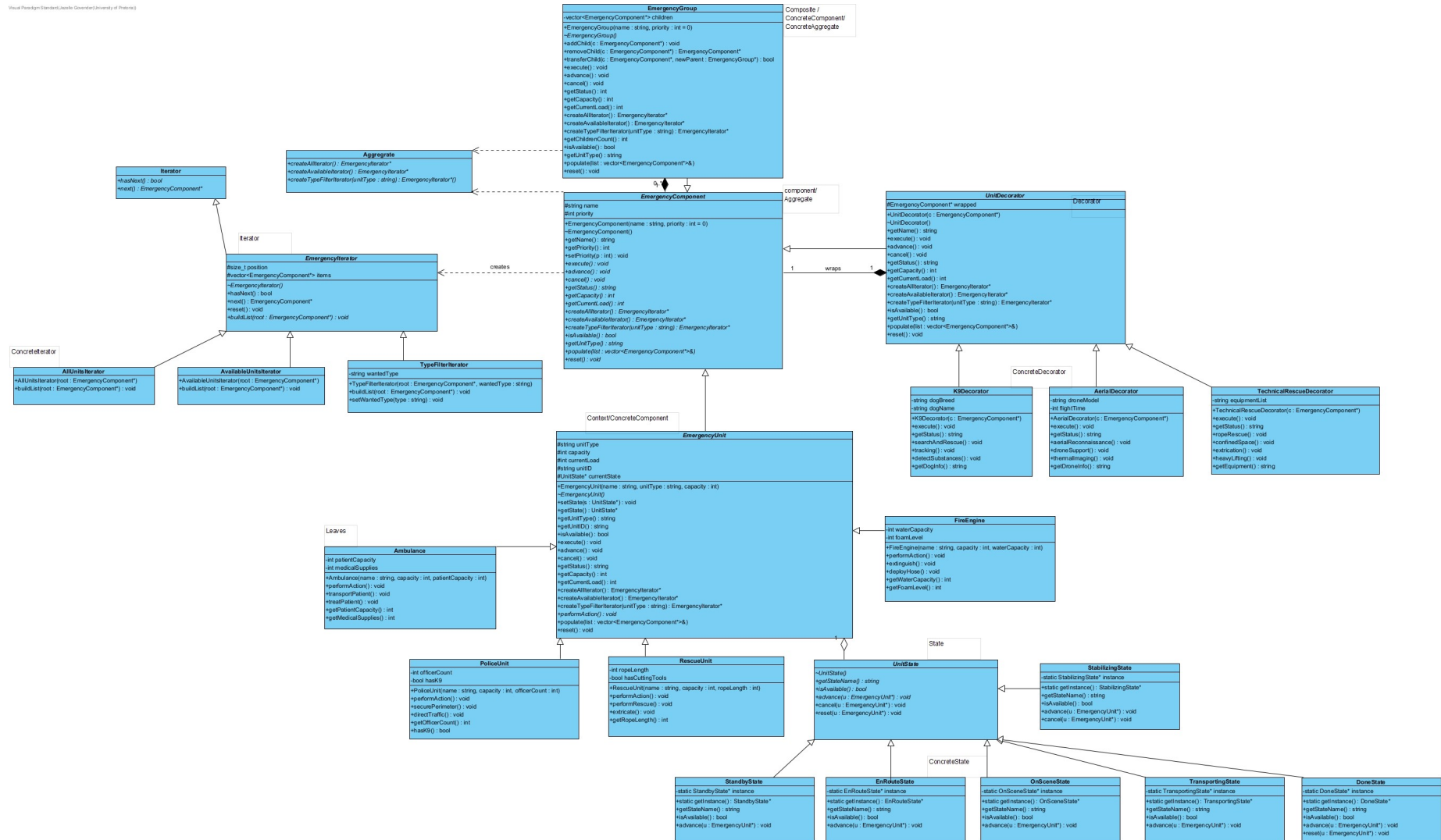
Now all four patterns work together through these three operations.

Decision 5: Ownership Rules

Who	Owns What	Deletes What
EmergencyGroup	children vector	All children in destructor
UnitDecorator	wrapped pointer	wrapped in destructor
EmergencyUnit	currentState pointer	NOTHING - states are singletons, don't delete
Client (main.cpp)	EmergencyIterator*	Must delete after use

2. UML class diagram

Complete class diagram drawn in Visual Paradigm. Participant labels on the diagram: Composite / ConcreteComponent / ConcreteAggregate (EmergencyGroup), Component / Aggregate (EmergencyComponent), Context / ConcreteComponent (EmergencyUnit), Leaves (FireEngine, Ambulance, PoliceUnit, RescueUnit), Decorator (UnitDecorator), ConcreteDecorator (K9Decorator, AerialDecorator, TechnicalRescueDecorator), Iterator (EmergencyIterator), ConcreteIterator (AllUnitsIterator, AvailableUnitsIterator, TypeFilterIterator), State (UnitState), ConcreteState (the six state classes).



3. Design and ownership rationale

Ownership rules: EmergencyGroup owns its children and deletes them recursively in its destructor. UnitDecorator owns the component it wraps and deletes it, so a decorator chain is freed from the outermost layer inward. UnitState objects are singletons shared by every unit; no unit owns its state and none is ever deleted. Iterators are owned by the client code that creates them and hold non-owning pointers into the tree. removeChild() returns the removed component to the caller rather than deleting it, so the caller decides whether to re-add it (for example wrapped in a decorator) or delete it.

A single EmergencyGroup class is used for every level of the hierarchy rather than separate Section, Team and Command classes. This is what makes the Composite genuinely recursive: a group can hold groups to any depth, and getCapacity(), getCurrentLoad(), advance(), execute(), cancel() and reset() on a group simply forward to every child, whether that child is a unit, a decorated unit or another group.

isAvailable(), getUnitType() and populate() are declared on EmergencyComponent so that the Iterator, Composite and Decorator can collaborate without type checks or dynamic_cast. populate() is the hook the iterators use to build their snapshot: a group recurses into its children, a unit adds itself, a decorator forwards to the component it wraps. Client code never receives the children vector.

Implementation notes (DESIGN_NOTES.md)

Task 2 — Notes on the implementation

Bugs fixed in the Visual Paradigm-generated stub (not just TODOs filled in)

1. Private inheritance everywhere it should be public: EmergencyUnit, FireEngine, Ambulance, PoliceUnit, RescueUnit, EmergencyIterator, AllUnitsIterator, AvailableUnitsIterator, TypeFilterIterator. Visual Paradigm's default C++ export uses private inheritance unless you set it explicitly on the diagram. Left as-is, none of these could be stored/used through their base class pointer — the Composite and Iterator patterns would not have compiled the way the design needs.
2. EmergencyGroup was missing its children vector — declared in the UML, absent from the generated header.
3. Singleton pattern was broken on every State class: generated code declared a non-static method literally named static_getInstance, which is not how you write a static factory method in C++. Rewrote each with a private constructor, a private static X* instance;, and a real static X* getInstance();.
4. PoliceUnit had a field and a method both named hasK9 — flat compile error. Renamed the field to hasK9Unit.
5. Mangled field names: Ambulance's int_patientCapacity/int_medicalSupplies → patientCapacity/medicalSupplies. TypeFilterIterator's int_string_wantedType → string wantedType.
6. Dead code in UnitState: every concrete state re-declared a non-virtual cancel() that could never actually override anything, since the base declaration also wasn't virtual. Made the base version virtual with one shared implementation and removed the pointless redeclarations.
7. Header include cycle: fixed by forward-declaring EmergencyComponent/ EmergencyIterator wherever only a pointer is used, instead of fully including.

Deviation from the UML you'll want to mirror in the diagram (Rule 9)

EmergencyComponent gained three methods not on the current diagram: - bool isAvailable() - string getUnitType() - void populate(vector<EmergencyComponent*>& list)

Why: Composite + Iterator + Decorator can only cooperate here without a single dynamic_cast or type-check (which Rule 8 forbids) if the base Component interface exposes whatever the traversal needs to ask of any node — leaf, group, or decorated leaf — without knowing which one it actually is. populate() is the actual Composite/Iterator handshake: a group recurses into its children; a leaf or

decorated item adds itself. No iterator ever touches EmergencyGroup's internal vector directly (Rule 7).

Traversal-modification policy (needed for Task 3/4 write-up)

Snapshot traversal. buildList() walks the tree once, at iterator-construction time, and copies the relevant pointers into the iterator's own vector. A later addChild/removeChild/transferChild on the live tree does not retroactively affect an iterator that already exists — it keeps iterating its original snapshot. Consequence: don't delete a component while a live iterator might still hold a pointer to it; finish or discard the iterator first.

Valgrind

still reachable: 48 bytes in 6 blocks, definitely lost: 0. The 6 blocks are the state singletons — never deleted, by design (Task 1, ownership rule: singletons have no per-instance data, so they're created once and live for the process's lifetime). "Still reachable" (not "definitely lost") is the correct outcome for that design and is worth citing directly in Task 5.

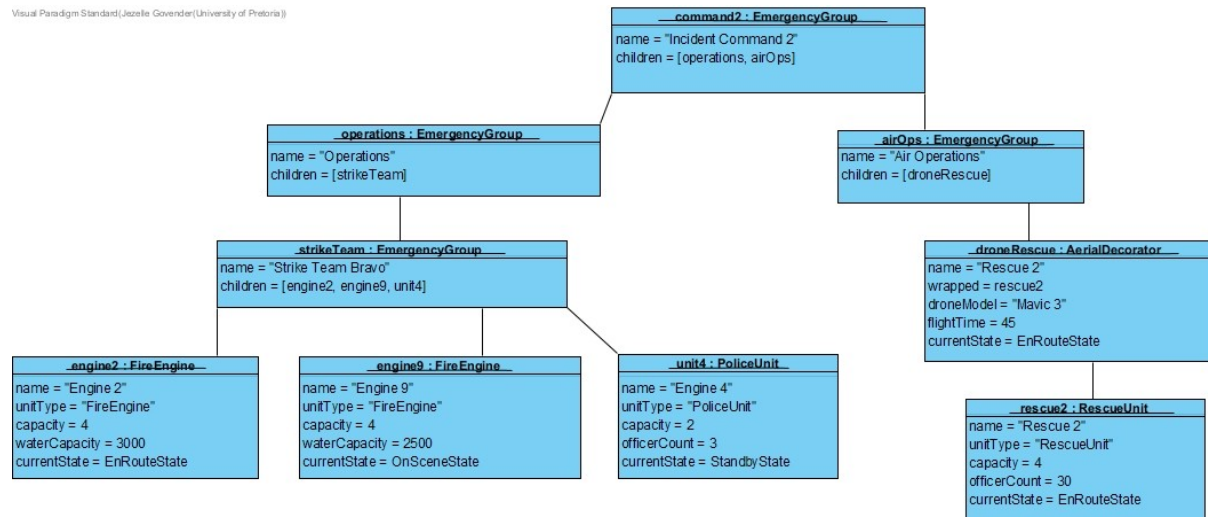
demo_main.cpp

This is a validation harness I used to prove the model works — 4-level hierarchy, two independent iterators, a type filter, full state lifecycle including one invalid transition, a stacked decorator, and a runtime structural change. It's not a Task 3 deliverable (that's not my assigned task) but it's a real working example your teammate doing Task 3 can borrow from or extend for the actual application scenarios.

4. Object diagram

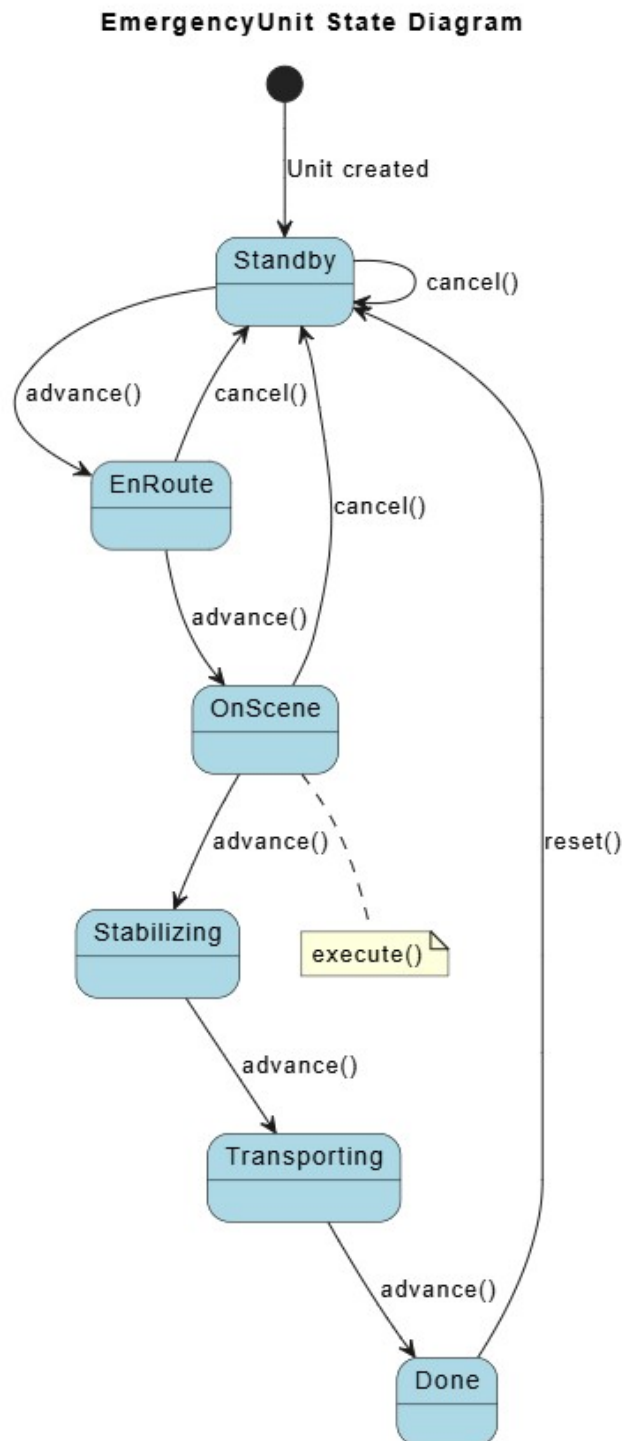
Runtime slice of the incident 2 (veld fire) hierarchy: Incident Command 2 holding Operations and Air Operations, Operations holding Strike Team Bravo after Unit 4 has been transferred into it, and Air Operations holding the decorated Rescue 2.

Visual Paradigm Standard (Jezelle Govender (University of Pretoria))



5. State diagram

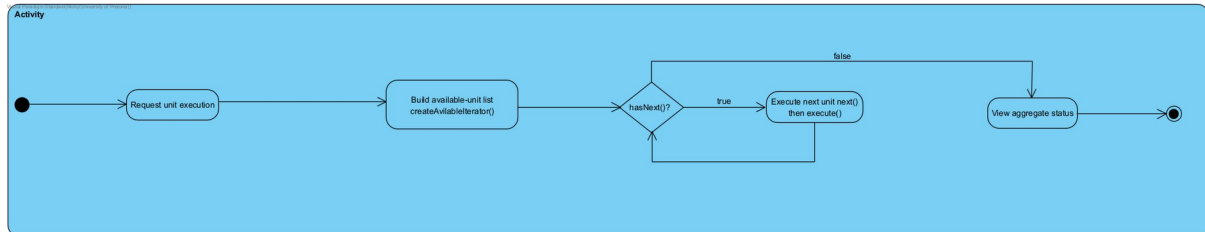
Lifecycle of an EmergencyUnit. `advance()` moves a unit forward through Standby, En Route, On Scene, Stabilizing, Transporting and Done. `cancel()` ends the mission early. `reset()` returns a finished unit to Standby so it can be dispatched again. `execute()` performs the state-appropriate action without changing state. In the code, `advance()` on a Done unit and `reset()` on a unit that is not Done are refused and logged.



6. Activity diagrams

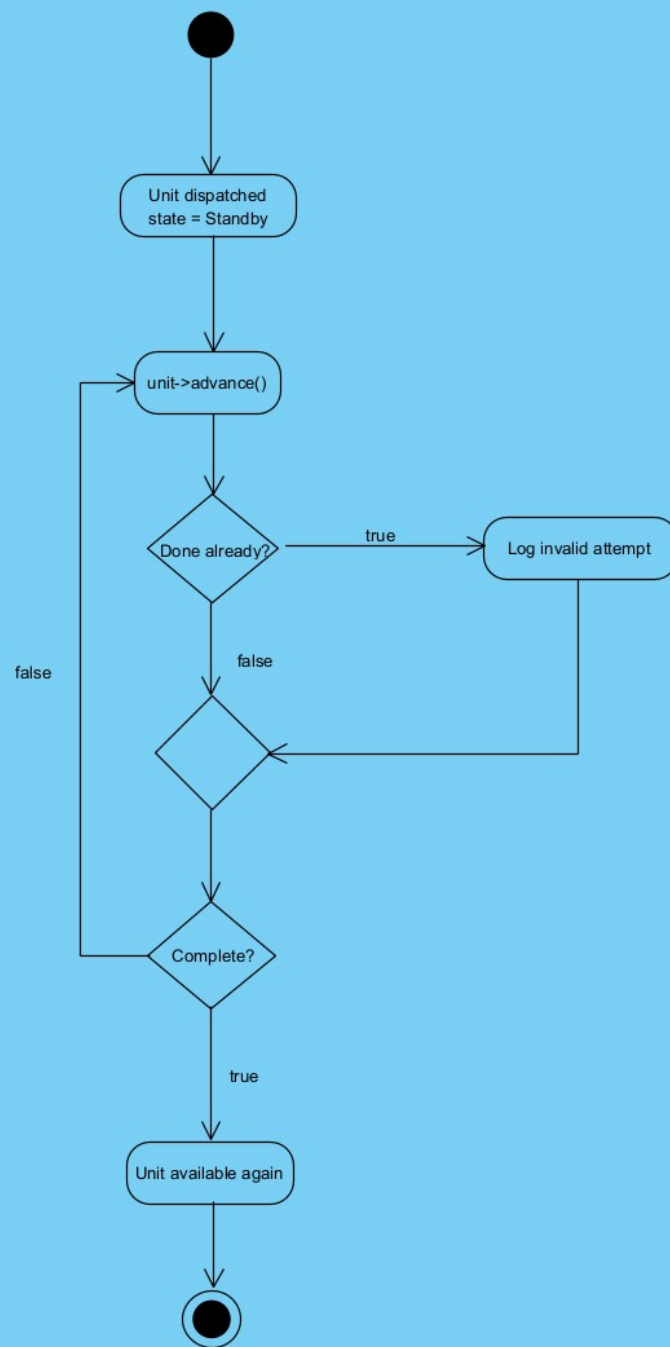
6.1 Dispatch and execute available units

Makes the iterator traversal visible as an explicit hasNext()/next() loop rather than a single process-all action. Traceable to the AvailableUnitsIterator usage in main.cpp.



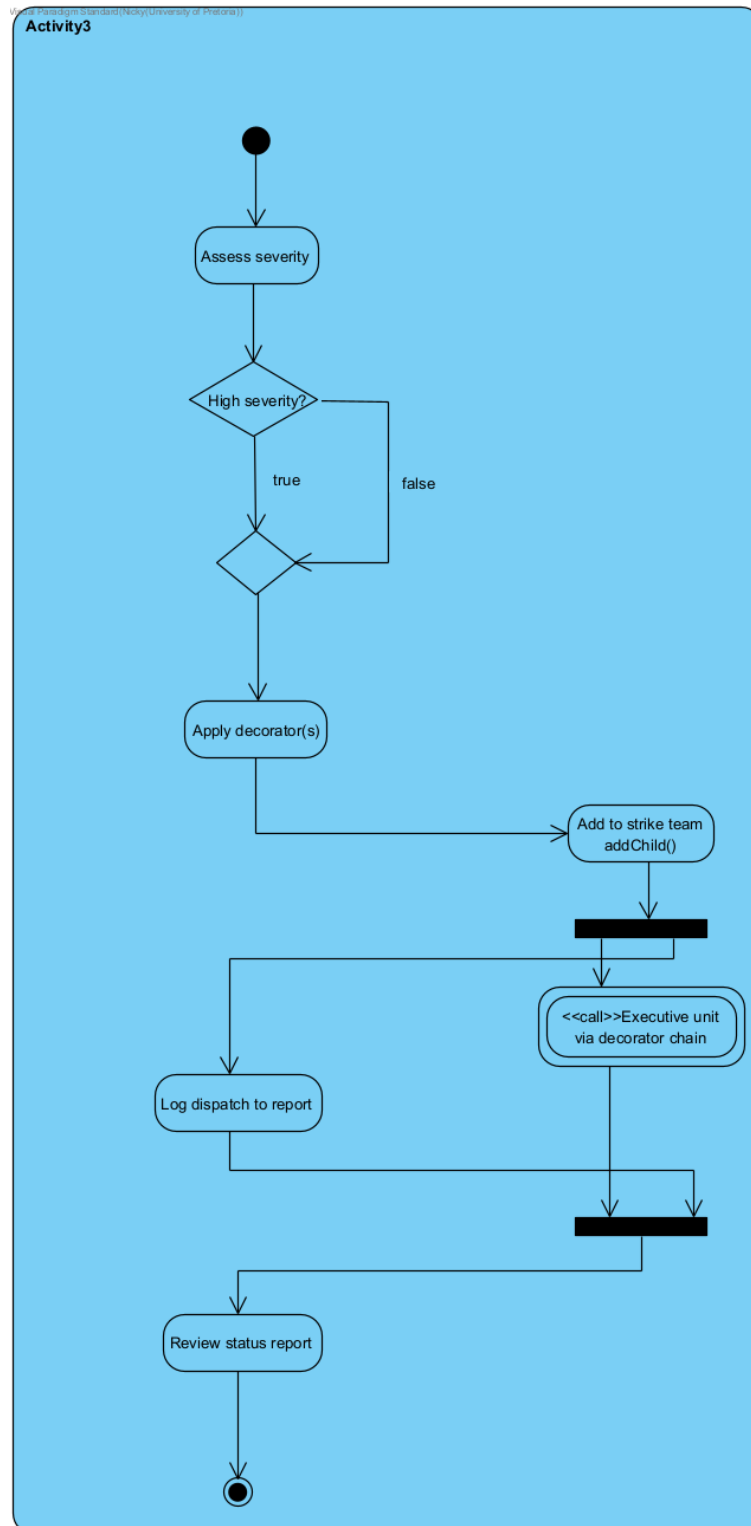
6.2 Unit lifecycle advance with guard

Conditional behaviour on the lifecycle: advance() is refused when the unit is already Done, otherwise the unit moves on until it completes and becomes available again. Traceable to UnitState::advance and DoneState::advance.



6.3 Decorated dispatch with structural reassignment

Decision on severity, decorators applied, unit added to a strike team, then a fork/join running decorated execution (called activity through the decorator chain) in parallel with logging. Traceable to the K9Decorator wrapping of Rescue 1 in incident 1 and the stacked AerialDecorator/TechnicalRescueDecorator on Rescue 2 in incident 2.



7. Runtime scenarios (Task 3)

main.cpp runs two incidents in sequence. Each builds its own Incident Command tree, works the incident, and deletes the tree. The domain summary of each is in docs/Incidents for main.docx. This section maps them to the Task 3 requirements.

Incident 1: structural collapse. The tree is Incident Command > Operations > Strike Team Alpha > units, with a Medical Branch beside Operations (three levels below the root). Traversal: an AvailableUnitsIterator lists which units are free after Strike Team and Unit 12 are dispatched. State: Strike Team advances as a group, Medic 3 is cancelled from En Route, Engine 7 runs Standby to Done, is refused a further advance, and is reset; a reset on Medic 5 while on scene is refused. Decoration: Rescue 1 is removed from Strike Team, wrapped in a K9Decorator and re-added while the incident is running, then executes through the decorator. Structural change: the remove and re-add of Rescue 1.

Incident 2: veld fire. The tree is Incident Command > Operations > Strike Team Bravo > units, with an Air Operations branch holding a doubly decorated unit (AerialDecorator over TechnicalRescueDecorator over Rescue 2). Traversal: an AllUnitsIterator over Strike Team Bravo is created before Unit 4 is transferred in, then compared with one created after; a TypeFilterIterator advances every FireEngine in the tree in one pass; an AvailableUnitsIterator shows Engine 9 back in the pool after it is cancelled. State: Air Ops and Strike Team advance and execute as groups. Decoration: the stacked decorator executes and both layers add their behaviour. Structural change: transferChild of Unit 4 from Operations into Strike Team Bravo, with the traversal snapshot policy demonstrated on it.

Between the two incidents every requirement in Task 3 is covered at least once, and the structural change in incident 2 is the one used to demonstrate the traversal-modification policy.

8. Traversal-modification policy

Policy: snapshot traversal.

An iterator copies the pointers it needs into its own vector when it is created, by calling populate() on the root. After that it never reads the tree again. Adding, removing or transferring a component does not affect an iterator that already exists; a new iterator sees the new structure. Incident 2 shows this with a roster of Strike Team Bravo is taken, Unit 4 is moved into the team, and the old iterator still lists two units while a new one lists three. Because the snapshot holds live pointers, state and decoration changes are still visible through an existing iterator. Only membership is frozen.

The one rule for client code is not to delete a component while an iterator that captured it is still in use. removeChild() returns the removed component instead of deleting it, so the caller controls when that happens. In the scenarios every iterator is deleted before the tree is.

Snapshot was chosen over live traversal or invalidation because it is the simplest policy that is always safe: no skipped or repeated elements, no dangling positions after a resize, and two iterators over the same tree cannot interfere with each other.

9. Debugging and memory investigation (Task 5)

9.1 GDB investigation of the Composite recursion

GDB was used inside the Docker container (which provides g++, make, gdb and valgrind) to confirm that EmergencyGroup::getCapacity() recurses correctly through nested groups and that decorated units report their capacity through the decorator chain. The binary is built with -g by the Makefile so source lines and variable names are available.

A breakpoint was set on EmergencyGroup::getCapacity and the program run until the final capacity total of incident 2 is computed (main.cpp line 155). The breakpoint was hit once per group. The backtrace at the third hit shows three getCapacity frames stacked, each with a different this pointer, all returning to line 90 (total += children[i]->getCapacity()). That is the Composite pattern recursion: Incident Command 2 calls Operations, which calls Strike Team Bravo.

```
#0 EmergencyGroup::getCapacity (this=0x5af0c9019790) at EmergencyGroup.cpp:88
#1 EmergencyGroup::getCapacity (this=0x5af0c9019850) at EmergencyGroup.cpp:90
#2 EmergencyGroup::getCapacity (this=0x5af0c90192f0) at EmergencyGroup.cpp:90
#3 wildfire (command=0x5af0c90192f0) at main.cpp:155
#4 main () at main.cpp:165
```

The fourth hit is the Air Operations group, called directly from the root, so the stack is only two frames deep. This shows the tree has uneven depth. print name confirmed which group was being processed and finish ran the call to completion, returning 4, which is the capacity of Rescue 2 passed up through the TechnicalRescueDecorator and AerialDecorator without either decorator changing it.

```
(gdb) print name
$1 = "Air Operations"
(gdb) finish
Value returned is $2 = 4
```

The full session is in docs/gdb_session.txt and reproduced in section 9.4.

9.2 Bug encountered during development

Symptom

While writing the incident scenarios for Task 3, the output of EmergencyGroup::getStatus() for a nested hierarchy was wrong. Every group and unit was printed at the same indentation regardless of depth, and blank lines appeared between entries. For a three level tree the output looked like this:

```
[Group: Incident Command] (2 children)
  [Group: Operations] (1 children)
    [Group: Strike Team Alpha] (3 children)
      [Unit FireEngine-1 "Engine 7" (FireEngine)] state=Done load=0/4
      [Unit RescueUnit-4 "Rescue 1" (RescueUnit)] state=Standby load=0/4 +K9[Rex/German Shepherd]
      [Unit PoliceUnit-3 "Unit 12" (PoliceUnit)] state=Standby load=0/2

    [Unit Ambulance-2 "Medic 3" (Ambulance)] state=Standby load=0/2 ...
```

Strike Team Alpha is inside Operations but is printed at the same level, and the units inside Strike Team are printed at the same level as the groups. The structure was correct (transferChild had moved Unit 12 into Strike Team and getChildrenCount confirmed it) but the display did not show it.

Cause

The original implementation prefixed two spaces to the whole string returned by each child and then appended a newline:

```
for (size_t i = 0; i < children.size(); i++) {
    oss << "  " << children[i]->getStatus() << "\n";
}
```

When the child is itself a group, `getStatus()` returns a multi-line string. The two space prefix is only applied to the first line of that string, so every line after the first loses its indentation. The trailing newline is added on top of the newline the child group already ended with, which produced the blank lines. The recursion was correct; the formatting did not recurse with it.

Debugging evidence

A breakpoint on `EmergencyGroup::getStatus` was used to inspect the string a child returns before the parent adds its prefix. For the Operations group the value of `children[i]->getStatus()` was a single string containing embedded newline characters, which confirmed the string was being treated as one line by the parent:

```
(gdb) break EmergencyGroup::getStatus
(gdb) run
(gdb) print name
$1 = "Incident Command"
(gdb) print children[0]->getStatus()
$2 = "[Group: Operations] (1 children)\n [Group: Strike Team Alpha] (3 children)\n
[Unit ..."
```

Correction

The child string is now split on newlines and each line is indented individually, so the indentation accumulates with depth. The newline is written before each line rather than after, which removes the duplicate blank lines. The method signature is unchanged, so the class diagram did not need to change.

```
for (size_t i = 0; i < children.size(); i++)
{
    istringstream lines(children[i]->getStatus());
    string line;
    while (getline(lines, line))
    {
        oss << "\n  " << line;
    }
}
```

Output after the fix:

```
[Group: Incident Command 2] (2 children)
[Group: Operations] (1 children)
  [Group: Strike Team Bravo] (3 children)
    [Unit FireEngine-6 "Engine 2" (FireEngine)] state=On Scene load=0/4
    [Unit FireEngine-7 "Engine 9" (FireEngine)] state=Done load=0/4
    [Unit PoliceUnit-8 "Unit 4" (PoliceUnit)] state=En Route load=0/2
  [Group: Air Operations] (1 children)
    [Unit RescueUnit-9 "Rescue 2" (RescueUnit)] state=On Scene load=0/4 +TechRescue[...]
+Aerial[...]
```

9.3 Valgrind evidence

Valgrind was run on the final application inside the Docker container with:

```
valgrind --leak-check=full --show-leak-kinds=all ./taskforge
```

Result (full output in docs/valgrind_output.txt):

```
HEAP SUMMARY:
  in use at exit: 48 bytes in 6 blocks
LEAK SUMMARY:
  definitely lost: 0 bytes in 0 blocks
  indirectly lost: 0 bytes in 0 blocks
  possibly lost: 0 bytes in 0 blocks
  still reachable: 48 bytes in 6 blocks
ERROR SUMMARY: 0 errors from 0 contexts
```

There are no memory errors and nothing definitely or indirectly lost. The 48 bytes still reachable are the six UnitState singletons (StandbyState, EnRouteState, OnSceneState, StabilizingState, TransportingState, DoneState), 8 bytes each. Each stack trace in the Valgrind output ends in a *State::getInstance() call. These objects are created once on first use and live until the process exits. They hold no per-unit data and no EmergencyUnit owns them, which is why EmergencyUnit::~EmergencyUnit() does not delete currentState. Still reachable is the expected result for this ownership decision and is not a leak.

Everything else in the program is freed: EmergencyGroup deletes its children recursively, UnitDecorator deletes the component it wraps, and every iterator is deleted by the code that created it. Deleting the root Incident Command group therefore frees the entire tree, including decorator chains.

9.4 Full GDB session (docs/gdb_session.txt)

```
Breakpoint 1, EmergencyGroup::getCapacity (this=0x5af0c9019850) at EmergencyGroup.cpp:88
88         int total = 0;
(gdb) bt
#0  EmergencyGroup::getCapacity (this=0x5af0c9019850) at EmergencyGroup.cpp:88
#1  0x00005af0a4fdb794 in EmergencyGroup::getCapacity (this=0x5af0c90192f0) at
EmergencyGroup.cpp:90
#2  0x00005af0a4fe1517 in wildfire (command=0x5af0c90192f0) at main.cpp:155
#3  0x00005af0a4fe1a77 in main () at main.cpp:165
(gdb) continue
Continuing.
```

```
Breakpoint 1, EmergencyGroup::getCapacity (this=0x5af0c9019790) at EmergencyGroup.cpp:88
88         int total = 0;
(gdb) bt
#0  EmergencyGroup::getCapacity (this=0x5af0c9019790) at EmergencyGroup.cpp:88
#1  0x00005af0a4fdb794 in EmergencyGroup::getCapacity (this=0x5af0c9019850) at
EmergencyGroup.cpp:90
#2  0x00005af0a4fdb794 in EmergencyGroup::getCapacity (this=0x5af0c90192f0) at
EmergencyGroup.cpp:90
#3  0x00005af0a4fe1517 in wildfire (command=0x5af0c90192f0) at main.cpp:155
#4  0x00005af0a4fe1a77 in main () at main.cpp:165
(gdb) continue
Continuing.
```

```
Breakpoint 1, EmergencyGroup::getCapacity (this=0x5af0c90197e0) at EmergencyGroup.cpp:88
88         int total = 0;
(gdb) bt
#0  EmergencyGroup::getCapacity (this=0x5af0c90197e0) at EmergencyGroup.cpp:88
#1  0x00005af0a4fdb794 in EmergencyGroup::getCapacity (this=0x5af0c90192f0) at
EmergencyGroup.cpp:90
#2  0x00005af0a4fe1517 in wildfire (command=0x5af0c90192f0) at main.cpp:155
#3  0x00005af0a4fe1a77 in main () at main.cpp:165
(gdb) print name
$1 = "Air Operations"
(gdb) finish
Run till exit from #0  EmergencyGroup::getCapacity (this=0x5af0c90197e0) at
EmergencyGroup.cpp:88
0x00005af0a4fdb794 in EmergencyGroup::getCapacity (this=0x5af0c90192f0) at
EmergencyGroup.cpp:90
```

```
90          total += children[i]->getCapacity();  
Value returned is $2 = 4  
(gdb) continue  
Continuing.  
Total capacity 14, current load 0
```

```
[Inferior 1 (process 24) exited normally]
```

9.5 Full Valgrind output (docs/valgrind_output.txt)

```
==82== Memcheck, a memory error detector
==82== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==82== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==82== Command: ./taskforge
==82==
--- Incident 1: structural collapse, Church Street ---
```

```
Call received. Resources assigned:
[Group: Incident Command 1] (2 children)
  [Group: Operations] (2 children)
    [Group: Strike Team Alpha] (2 children)
      [Unit FireEngine-1 "Engine 7" (FireEngine)] state=Standby load=0/4
      [Unit RescueUnit-2 "Rescue 1" (RescueUnit)] state=Standby load=0/4
    [Unit PoliceUnit-3 "Unit 12" (PoliceUnit)] state=Standby load=0/2
  [Group: Medical Branch] (2 children)
    [Unit Ambulance-4 "Medic 3" (Ambulance)] state=Standby load=0/2
    [Unit Ambulance-5 "Medic 5" (Ambulance)] state=Standby load=0/2
```

```
Dispatching Strike Team Alpha and Unit 12.
Units still available for other calls:
  [Unit Ambulance-4 "Medic 3" (Ambulance)] state=Standby load=0/2
  [Unit Ambulance-5 "Medic 5" (Ambulance)] state=Standby load=0/2
```

```
Strike Team arrives on scene.
FireEngine-1 deploying hose (water capacity 2000L)
FireEngine-1 extinguishing fire (foam level now 90)
RescueUnit-2 performing rescue (rope length 50m)
RescueUnit-2 extricating casualty using cutting tools
```

```
Survivors reported trapped. K9 team attached to Rescue 1.
RescueUnit-2 performing rescue (rope length 50m)
RescueUnit-2 extricating casualty using cutting tools
  (K9 Rex) searching for survivors
```

```
Medic 3 dispatched, then stood down when Medic 5 is closer.
[Cancel] Ambulance-4 mission cancelled while En Route.
Ambulance-5 treating patient (supplies left 90)
Ambulance-5 transporting patient (capacity 2)
```

```
Engine 7 completes its work and returns to the pool.
[Invalid] FireEngine-1 is already Done; cannot advance further.
[Reset] FireEngine-1 returned to the available units.
```

```
Medic 5 is still on scene, Medic 3 is stood down.
[Invalid] Ambulance-5 is On Scene;cancel it before resetting.
[Reset] Ambulance-4 returned to the available units.
```

```
End of incident 1:
[Group: Incident Command 1] (2 children)
  [Group: Operations] (2 children)
    [Group: Strike Team Alpha] (2 children)
      [Unit FireEngine-1 "Engine 7" (FireEngine)] state=Standby load=0/4
      [Unit RescueUnit-2 "Rescue 1" (RescueUnit)] state=On Scene load=0/4 +K9[Rex/German Shepherd]
    [Unit PoliceUnit-3 "Unit 12" (PoliceUnit)] state=En Route load=0/2
  [Group: Medical Branch] (2 children)
    [Unit Ambulance-4 "Medic 3" (Ambulance)] state=Standby load=0/2
    [Unit Ambulance-5 "Medic 5" (Ambulance)] state=On Scene load=0/2
```

```
--- Incident 2: veld fire, Bronkhorstspuit road ---
```

```
Initial deployment:
[Group: Incident Command 2] (2 children)
  [Group: Operations] (2 children)
    [Group: Strike Team Bravo] (2 children)
      [Unit FireEngine-6 "Engine 2" (FireEngine)] state=Standby load=0/4
      [Unit FireEngine-7 "Engine 9" (FireEngine)] state=Standby load=0/4
    [Unit PoliceUnit-8 "Unit 4" (PoliceUnit)] state=Standby load=0/2
```

```
[Group: Air Operations] (1 children)
  [Unit RescueUnit-9 "Rescue 2" (RescueUnit)] state=Standby load=0/4 +TechRescue[hydraulic
spreaders, rope k
```

Fire jumps the road. Unit 4 reassigned from Operations into Strike Team Bravo.
Operations now has 1 child, Strike Team Bravo has 3.

```
Strike Team Bravo iterator created before the move (snapshot):
  [Unit FireEngine-6 "Engine 2" (FireEngine)] state=Standby load=0/4
  [Unit FireEngine-7 "Engine 9" (FireEngine)] state=Standby load=0/4
```

```
Strike Team Bravo iterator created after the move:
  [Unit FireEngine-6 "Engine 2" (FireEngine)] state=Standby load=0/4
  [Unit FireEngine-7 "Engine 9" (FireEngine)] state=Standby load=0/4
  [Unit PoliceUnit-8 "Unit 4" (PoliceUnit)] state=Standby load=0/2
```

All engines ordered to advance together.

```
Fire engines:
  [Unit FireEngine-6 "Engine 2" (FireEngine)] state=En Route load=0/4
  [Unit FireEngine-7 "Engine 9" (FireEngine)] state=En Route load=0/4
```

Air Operations conducts reconnaissance while ground crews work.
RescueUnit-9 performing rescue (rope length 30m)
RescueUnit-9 extricating casualty using cutting tools
(TechRescue) heavy extrication using hydraulic spreaders, rope kit
(Drone DJI-Matrice) conducting aerial reconnaissance

Strike Team Bravo on scene and working.
FireEngine-6 deploying hose (water capacity 3000L)
FireEngine-6 extinguishing fire (foam level now 90)
FireEngine-7 deploying hose (water capacity 2500L)
FireEngine-7 extinguishing fire (foam level now 90)
PoliceUnit-8 securing perimeter with 3 officers
PoliceUnit-8 directing traffic

Engine 9 is withdrawn to cover a second call.
[Cancel] FireEngine-7 mission cancelled while On Scene.
Units available for the second call:
 [Unit FireEngine-7 "Engine 9" (FireEngine)] state=Done load=0/4

```
End of incident 2:
[Group: Incident Command 2] (2 children)
  [Group: Operations] (1 children)
    [Group: Strike Team Bravo] (3 children)
      [Unit FireEngine-6 "Engine 2" (FireEngine)] state=On Scene load=0/4
      [Unit FireEngine-7 "Engine 9" (FireEngine)] state=Done load=0/4
      [Unit PoliceUnit-8 "Unit 4" (PoliceUnit)] state=En Route load=0/2
    [Group: Air Operations] (1 children)
      [Unit RescueUnit-9 "Rescue 2" (RescueUnit)] state=On Scene load=0/4 +TechRescue[hydraulic
spreaders, rope
Total capacity 14, current load 0
```

```
==82==
==82== HEAP SUMMARY:
==82==    in use at exit: 48 bytes in 6 blocks
==82==   total heap usage: 262 allocs, 256 frees, 119,406 bytes allocated
==82==
==82== 8 bytes in 1 blocks are still reachable in loss record 1 of 6
==82==    at 0x4849013: operator new(unsigned long) (in
/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.s
==82==    by 0x112678: StandbyState::getInstance() (StandbyState.cpp:9)
==82==    by 0x1102FF: EmergencyUnit::EmergencyUnit(std::__cxx11::basic_string<char,
std::char_traits<char>, s
==82==    by 0x111292: FireEngine::FireEngine(std::__cxx11::basic_string<char,
std::char_traits<char>, std::al
==82==    by 0x113F58: buildingCollapse(EmergencyGroup*) (main.cpp:32)
==82==    by 0x1159EF: main (main.cpp:161)
==82==
==82== 8 bytes in 1 blocks are still reachable in loss record 2 of 6
==82==    at 0x4849013: operator new(unsigned long) (in
/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.s
```

```

==82==    by 0x11101C: EnRouteState::getInstance() (EnRouteState.cpp:9)
==82==    by 0x11275D: StandbyState::advance(EmergencyUnit*) (StandbyState.cpp:23)
==82==    by 0x110685: EmergencyUnit::advance() (EmergencyUnit.cpp:65)
==82==    by 0x10F3BC: EmergencyGroup::advance() (EmergencyGroup.cpp:53)
==82==    by 0x1142C8: buildingCollapse(EmergencyGroup*) (main.cpp:51)
==82==    by 0x1159EF: main (main.cpp:161)
==82==
==82== 8 bytes in 1 blocks are still reachable in loss record 3 of 6
==82==    at 0x4849013: operator new(unsigned long) (in
/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.s
==82==    by 0x111BDA: OnSceneState::getInstance() (OnSceneState.cpp:9)
==82==    by 0x111101: EnRouteState::advance(EmergencyUnit*) (EnRouteState.cpp:23)
==82==    by 0x110685: EmergencyUnit::advance() (EmergencyUnit.cpp:65)
==82==    by 0x10F3BC: EmergencyGroup::advance() (EmergencyGroup.cpp:53)
==82==    by 0x11439D: buildingCollapse(EmergencyGroup*) (main.cpp:56)
==82==    by 0x1159EF: main (main.cpp:161)
==82==
==82== 8 bytes in 1 blocks are still reachable in loss record 4 of 6
==82==    at 0x4849013: operator new(unsigned long) (in
/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.s
==82==    by 0x110E9FE: DoneState::getInstance() (DoneState.cpp:11)
==82==    by 0x1105CF: EmergencyUnit::execute() (EmergencyUnit.cpp:56)
==82==    by 0x10F354: EmergencyGroup::execute() (EmergencyGroup.cpp:47)
==82==    by 0x1143BA: buildingCollapse(EmergencyGroup*) (main.cpp:57)
==82==    by 0x1159EF: main (main.cpp:161)
==82==
==82== 8 bytes in 1 blocks are still reachable in loss record 5 of 6
==82==    at 0x4849013: operator new(unsigned long) (in
/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.s
==82==    by 0x112468: StabilizingState::getInstance() (StabilizingState.cpp:9)
==82==    by 0x111CBF: OnSceneState::advance(EmergencyUnit*) (OnSceneState.cpp:23)
==82==    by 0x110685: EmergencyUnit::advance() (EmergencyUnit.cpp:65)
==82==    by 0x114577: buildingCollapse(EmergencyGroup*) (main.cpp:76)
==82==    by 0x1159EF: main (main.cpp:161)
==82==
==82== 8 bytes in 1 blocks are still reachable in loss record 6 of 6
==82==    at 0x4849013: operator new(unsigned long) (in
/usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.s
==82==    by 0x112CBC: TransportingState::getInstance() (TransportingState.cpp:9)
==82==    by 0x11254D: StabilizingState::advance(EmergencyUnit*) (StabilizingState.cpp:23)
==82==    by 0x110685: EmergencyUnit::advance() (EmergencyUnit.cpp:65)
==82==    by 0x11458E: buildingCollapse(EmergencyGroup*) (main.cpp:77)
==82==    by 0x1159EF: main (main.cpp:161)
==82==
==82== LEAK SUMMARY:
==82==    definitely lost: 0 bytes in 0 blocks
==82==    indirectly lost: 0 bytes in 0 blocks
==82==    possibly lost: 0 bytes in 0 blocks
==82==    still reachable: 48 bytes in 6 blocks
==82==    suppressed: 0 bytes in 0 blocks
==82==
==82== For lists of detected and suppressed errors, rerun with: -s
==82== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

```

10. GitHub repository and workflow reflection

Repository: <https://github.com/<org>/Prac4-COS214>

Work was split by task onto feature branches: feature/core-implementation (Nicole: core model), feature/uml-diagrams (Jezelle: design and diagrams) and feature/dynamic-behaviour (Rudolph: reset(), scenarios, Docker, README, debugging evidence). Each branch was merged into main through pull requests. Diagram and code changes were coordinated through a written list of UML edits so that the class diagram and the implementation were changed to match rather than drifting apart. The main friction was keeping local copies in sync: on two occasions an older copy of a header was pushed over a newer one and broke the build until restored; pulling before pushing and building before every commit fixed this. Example activity: <commit hash> added reset() across the hierarchy; <commit hash> replaced Testingmain.cpp with the scenario main.cpp; PR #<n> merged the Docker environment; PR #<n> merged the corrected class diagram.

11. Contribution statement

Nicole's Contributions:

Task 2: Core Model Implementation

Bug fixes to the generated stub: Corrected private inheritance on EmergencyUnit, all four leaf classes, and the entire Iterator hierarchy (would have blocked polymorphic use of the Composite/Iterator patterns entirely). Fixed a broken singleton pattern across all six State classes (the generated getInstance() methods weren't actually static). Resolved a field/method name collision in PoliceUnit that was a straight compile error, mangled field names in Ambulance and TypeFilterIterator, a missing children vector in EmergencyGroup, and a circular header include once EmergencyComponent was made to properly implement Aggregate.

Implemented the full core model: EmergencyComponent/EmergencyGroup/EmergencyUnit (Composite), all six concrete UnitState classes with valid and invalid transitions (State), UnitDecorator plus K9Decorator/AerialDecorator/TechnicalRescueDecorator with verified stacking (Decorator), and EmergencyIterator/AllUnitsIterator/AvailableUnitsIterator/TypeFilterIterator (Iterator), plus the four concrete leaf units (FireEngine, Ambulance, PoliceUnit, RescueUnit).

Added isAvailable(), getUnitType(), and populate() to EmergencyComponent so Composite, Iterator, and Decorator can cooperate without any type-checks or dynamic_cast, and documented this as a deviation for the UML to mirror.

Documented and implemented the team's ownership rules (destructors deleting owned children/wrapped objects, singleton states never deleted) and the snapshot traversal-modification policy (iterators capture pointers at construction; later structural changes don't retroactively affect an existing iterator).

Verified correctness with a validation harness exercising the full hierarchy, both iterators, decorator stacking, the full state lifecycle including an invalid transition, and a runtime structural change (transferChild). Compiled clean with -std=c++11 -Wall -Wextra. Ran Valgrind: definitely lost: 0 bytes, still reachable: 48 bytes (the six state singletons, by design).

Made 12 incremental commits on feature/core-implementation, each a genuinely buildable increment (interfaces → iterators → states → composite → leaves → decorators → validation demo), which were then merged to main.

Task 4 Diagrams:

Three UML Activity Diagrams: "Dispatch and Execute Available Units" (makes the iterator traversal visible as an explicit loop rather than hiding it behind a single action, with swimlanes for

Dispatcher/System), "Unit Lifecycle Advance with Guard" (shows the state-machine's conditional/invalid-transition behavior with a repeat loop), and "Decorated Dispatch with Structural Reassignment" (fork/join for parallel decorated-execution and logging branches, plus a called/composite activity referencing the decorator delegation chain).

Jezelle's Contributions:

Task 1: Architectural Design & UML

Domain Description: Defined the Emergency Response Coordination System (ERCS) - a system that coordinates emergency units (fire engines, ambulances, police, rescue teams) during incidents like wildfires and building collapses.

UML Class Diagram: I created the complete UML class diagram in Visual Paradigm, including all classes, relationships, multiplicities, and operations for the Composite, State, Decorator, and Iterator patterns.

GoF Participant Mapping: I identified and documented all GoF participants for each pattern on the UML.

Header Files: I generated and committed all header files from Visual Paradigm, and had to add the proper includes and public inheritance.

I wrote the design rationale explaining:

Why a single EmergencyGroup class was used instead of multiple composite subclasses (true recursion).

Ownership rules: EmergencyGroup owns children, UnitDecorator owns wrapped, states are non-owned singletons.

Task 4 Diagrams:

UML Object Diagram: I created an object diagram showing a meaningful runtime portion of the nested hierarchy - specifically the wildfire.

UML State Diagram: Created a state diagram showing the lifecycle of an EmergencyUnit with states.

Rudolph contributions

Rudolph - contribution summary

Implemented the reset() transition across the hierarchy (UnitState, DoneState, EmergencyComponent, EmergencyUnit, EmergencyGroup, UnitDecorator) so units can return from Done to Standby, adding a second invalid transition. Fixed the EmergencyGroup::getStatus() indentation bug, cleaned up EmergencyUnit::execute() to compare state by pointer, and renamed Aggregate to Aggregate to match the diagram.

Wrote main.cpp: two story-driven incidents (structural collapse, veld fire) covering traversal, state changes, runtime decoration, a structural transfer and the snapshot-iterator demonstration, replacing the pattern-by-pattern test main.

Created the Makefile (taskforge, run, valgrind, gdb, clean targets), the Dockerfile (Ubuntu 22.04 with g++, make, gdb, valgrind), the README with build, run, GDB and Valgrind instructions, and .gitignore/.dockerignore. Set up Docker in WSL, built the image and ran the program, Valgrind and GDB inside the container.

Produced the Task 5 evidence: docs/valgrind_output.txt (0 errors, 0 bytes definitely lost, 48 bytes still reachable explained as the six state singletons) and docs/gdb_session.txt (three-level getCapacity recursion, print name, finish), plus the Task 5 write-up including the getStatus() bug investigation.

Wrote the traversal-modification policy, the Task 3 scenario-to-requirement mapping, and the GitHub reflection and contribution statement. Supplied the class-diagram correction list and state-diagram specification to Jezelle and the scenario flows to Nicole for the activity diagrams.

All three members reviewed the complete submission and can explain any part of it.